

Challenge:	The Infinite Mirror
Category:	Forensics / Scripting
Difficulty:	Medium
Tools Used:	Linux Terminal (WSL), unzip, grep, Bash Scripting

Summary

This challenge demonstrates the dangers of a recursive decompression bomb (zip bomb) and the importance of selective extraction. The payload consists of 10 nested .zip layers, each potentially containing massive junk files designed to exhaust disk space and crash automated forensics tools like binwalk. By utilizing a custom Bash loop to selectively extract only the nested archives and a strict regex search to bypass malformed decoys, the hidden flag can be safely recovered from the resulting server logs.

Problem Statement

A relic from B-7, sealed in a curse, A mirror that multiplies - each layer grows worse. Unfold it once, and a thousand more rise, A flood of illusions before your eyes. Within the chaos, the truth is confined, Buried in echoes, yet perfectly aligned. Break it with force, and you'll summon its wrath - But tread with care, and you'll uncover the path.

Steps to Solve

1. Access the working environment via the WSL terminal and navigate to the directory containing the initial payload:
cd /mnt/c/Users/aksha/Downloads
2. Create an isolated workspace to avoid cluttering the local machine and copy the initial payload (1.zip) into it:
mkdir -p flag_workspace && cp 1.zip flag_workspace/ && cd flag_workspace
3. Recognize the payload as a potential Zip Bomb. Avoid using standard recursive extraction tools (like binwalk -Me) or GUI tools, as they will blindly extract all massive junk files and risk system failure.
4. Automate a safe extraction using a Bash while loop. This script selectively targets only the nested .zip files, extracts them, and immediately removes the old archives to prevent disk exhaustion:
**while ls *.zip > /dev/null 2>&1; do
for z in *.zip; do
unzip -q "\$z"
rm "\$z"
done
done**
5. Once all layers are bypassed, 60 server logs remain in the directory. Identify that the creator has planted flawed decoys (e.g., misspellings or missing brackets) among the logs.

6. Execute a strict Regular Expression (Regex) search using grep to recursively scan all 60 logs, filter out the flawed decoys, and isolate the exact flag syntax:
grep -rhoE "Cruxhunt\[^\]\+\" .
7. The command successfully ignores the noise and outputs only the correct hidden flag to the terminal.

```
ghxst0sint@Akshay:/mnt/c/users/aksha/downloads$ mkdir -p flag_workspace1 &&
cp 1.zip flag_workspace1/ && cd flag_workspace1
ghxst0sint@Akshay:/mnt/c/users/aksha/downloads/flag_workspace1$ while ls *.zip
ip > /dev/null 2>&1; do
> for z in *.zip; do
> unzip -q "$z"
> rm "$z"
> done
> done
ghxst0sint@Akshay:/mnt/c/users/aksha/downloads/flag_workspace1$ grep -rhoE "
Cruxhunt\[^\]\+\"
grep: Unmatched [, [^, [:, [., or [=
ghxst0sint@Akshay:/mnt/c/users/aksha/downloads/flag_workspace1$ grep -rhoE "
Cruxhunt\[^\]\+\" .
Cruxhunt{m1n1stry_1ntrus10n_d3t3ct3d}
ghxst0sint@Akshay:/mnt/c/users/aksha/downloads/flag_workspace1$ |
```

Final FLAG: Cruxhunt{m1n1stry_1ntrus10n_d3t3ct3d}